Rent Hatch Car Rental: A–Z Architecture & Plan

Nov 10, 2026 · @Hanzla

1. Executive summary

Rent Hatch adds a car rental vertical for the US and the GCC (UAE: Dubai and Sharjah, Qatar, Oman) on the platform that already runs event rentals. A Dubai pilot (self-drive, daily and monthly) goes live around week 32 and GA follows around week 39. The US launch and the other service modes follow in waves after that. The plan assumes 2–3 engineers working with coding agents.

- **One platform, two verticals.** Logins, customers, payments, invoices, messaging, documents, audit and agents are shared. Event bookings keep their existing flow. Car rentals get their own booking flow, fields, rules and screens.
- **A modular monolith of bounded contexts.** Each context publishes a contract and hides its tables. Contexts talk through a transactional outbox, and CI enforces the boundaries.
- **Everything configurable within guardrails.** Settings are layered from platform down to a single booking. Rules are stored as data. Country legal minimums can only be made stricter. A confirmed booking keeps the terms it was sold under.
- **GCC realities are designed in:**
  - 3-decimal OMR, and VAT with immutable invoices;
  - Salik/Darb tolls and police fines that arrive weeks after return;
  - local card gateways (Stripe doesn't onboard Qatar businesses);
  - Arabic RTL, WhatsApp, delivery to hotels, monthly rentals;
  - UAE and Oman e-invoicing.
- **Agents are governed principals.** They use the same commands as staff, under per-tenant policy, approvals, budgets, a kill switch and full logs.
- **Two regional cells.** A US cell and a UAE cell, so GCC data stays in the region.

Decisions already made

#	Decision	Choice
1	Users (logins)	One shared table. Roles per vertical on the membership

#	Decision	Choice
2	Customers	One shared core. Car-only driver and ID data in car tables, encrypted
3	Orders	Thin shared header + separate detail tables per vertical, enforced by the database
4	Event rentals	Keep the existing flow unchanged
5	ID documents in WhatsApp/chat	Never accepted. Secure upload link in the platform only (platform guardrail)
6	Chauffeur and insurance replacement	In scope, with new staff roles and external partner roles
7	Marketplaces and Meta	In scope through a configurable channel hub
8	Document and licence lists per country	Ship researched lists marked "verify"; owner verifies later
9	Pilot data	Public-terms research and 16 reference cases now. Real golden cases once pilots onboard

Confirmed by the owner on 13 Nov 2026:

- the pilot is one Dubai operator, self-drive, daily and monthly rentals;
- one login works in every region;
- legal entities are not built now: one company and one currency per workspace;
- the shared order header goes ahead, with a prototype first.

2. Markets, competitors and positioning

US rental software is weak on GCC needs, and GCC tools are weak on modern UX, self-service and AI. Rent Hatch can own both markets in one product.

What each market needs

Market	Must-haves
UAE (Dubai, Sharjah)	Salik/Darb tolls and police/RTA fines matched to rentals; deposits held for weeks; VAT 5% invoices; Arabic; WhatsApp; delivery to hotels, homes and DXB; monthly rentals; luxury booked as a specific car; e-invoicing (PINT AE) from 2027

Market	Must-haves
Qatar (Doha)	Visitor licence rules (7 or 14 days, then IDP); credit-card deposits; no VAT today (verify); a local card gateway
Oman (Muscat)	OMR with 3 decimals; VAT 5%; Fawtara e-invoicing (phased from Aug 2026); cross-border NOC from the UAE
US	Young-driver and debit-card rules by state; airport fees; insurance-replacement direct billing; Turo/Getaround hosts; late-return grace rules

Competitors

Group	Players	Gap we exploit
US/global rental software	HQ Rental Software, Rent Centric, Navotar, TSD, Bluebird RentWorks, Thermeon, Wheelbase, Rentsyst, Coastr	Weak on GCC tolls, fines, VAT, 3-decimal currencies, Arabic, local gateways
GCC local	Caryaati, Easy Rent Pro, Yo!Rent, ERP builds (ERPNext, Odoo, SAP B1), custom agencies	Dated UX, little self-service, no APIs or AI
AI layers	CarCloud AI agent (Mar 2026), AiRentoSoft, Point of Rental phone agent	Bolted onto someone else's system, so they can't enforce its rules in the same transaction
Damage AI	UVeye (Hertz, Sixt), ProovStation (Sixt), Ravin (moved to insurers)	Customer backlash over automatic billing
Marketplaces	OneClickDrive, Rentalcars.com, Discover Cars, CarTrawler, Turo, Getaround	Channels to connect, not to compete with

Positioning

1. US and GCC in one product.
2. Agents inside the system of record, under rules the operator controls.
3. Fair, evidence-first damage handling on a phone (a person always confirms).
4. Post-rental money handled well: tolls, fines and damage collected with evidence the customer can see.
5. Events and cars on one platform, enabling cross-sell (e.g. a luxury car for a wedding).

3. Current repo: keep and fix

The foundations are strong and car rental reuses them. The coupling between modules must be fixed before a second vertical is added.

Stack: Next.js 16 + React 19 on Vercel (region pdx1, next to the database in Oregon); PostgreSQL 17 (Supabase) with Drizzle and row-level security; Better Auth; graphile-worker on Railway; pnpm + Turborepo; an Astro marketing site.

Keep

Area	What exists
Tenancy	Row-level security on every table, a non-owner app role, `withTenant()`, composite keys, 404-not-403, location scope enforced by the database
Pure core	`@rental/core` pricing engine (150/150 Odoo parity), versioned zod settings, permissions matrix, regional detection
Availability	Overlap refused in SQL under an advisory lock; `btree_gist` already installed
Price locks and versions	Saved prices never move; optimistic version check; order history
Messaging	Outbox with idempotency keys, scheduled sends, tenant SMTP/SMS
Money ops	Idempotent Stripe calls, refund planning, encrypted secrets. Reconciliation is polling; webhooks are missing
Field app	Coach portal (checklists, photos, GPS). It is online-only, so the offline field app is new work
Delivery discipline	CI on real PG17, nightly backups, branch per phase, e2e scripts, parity script

Fix

#	Finding	Fix
F1	`@rental/db` holds every table and service behind one barrel; `orders.ts` (1,241 lines) calls messaging, money, catalog, customers	Bounded contexts with contracts and an outbox
F2	Messaging and payments read tables directly	Contexts own their tables
F3	`orders` and roles are event-shaped	Shared order header + per-vertical tables; roles per vertical
F4	Availability is quantity per product	Pooled, class/model and serialized (VIN) strategies
F5	Money is `*_cents` (2 decimals)	Money with currency exponent (OMR = 3)
F6	Invoices are re-synced on every change	Issued invoices immutable; credit/debit notes
F7	One tax percent	Multi-component tax engine
F8	Stripe only, no holds	Gateway port with GCC adapters and deposit holds
F9	No i18n or RTL; no QA/OM defaults	ICU catalog, logical CSS, bilingual documents
F10	One US region	Regional cells
F11	No public API or agent tools	Contracts generate API, webhooks and MCP tools
F12	One worker queue	Named queues and per-tenant fairness

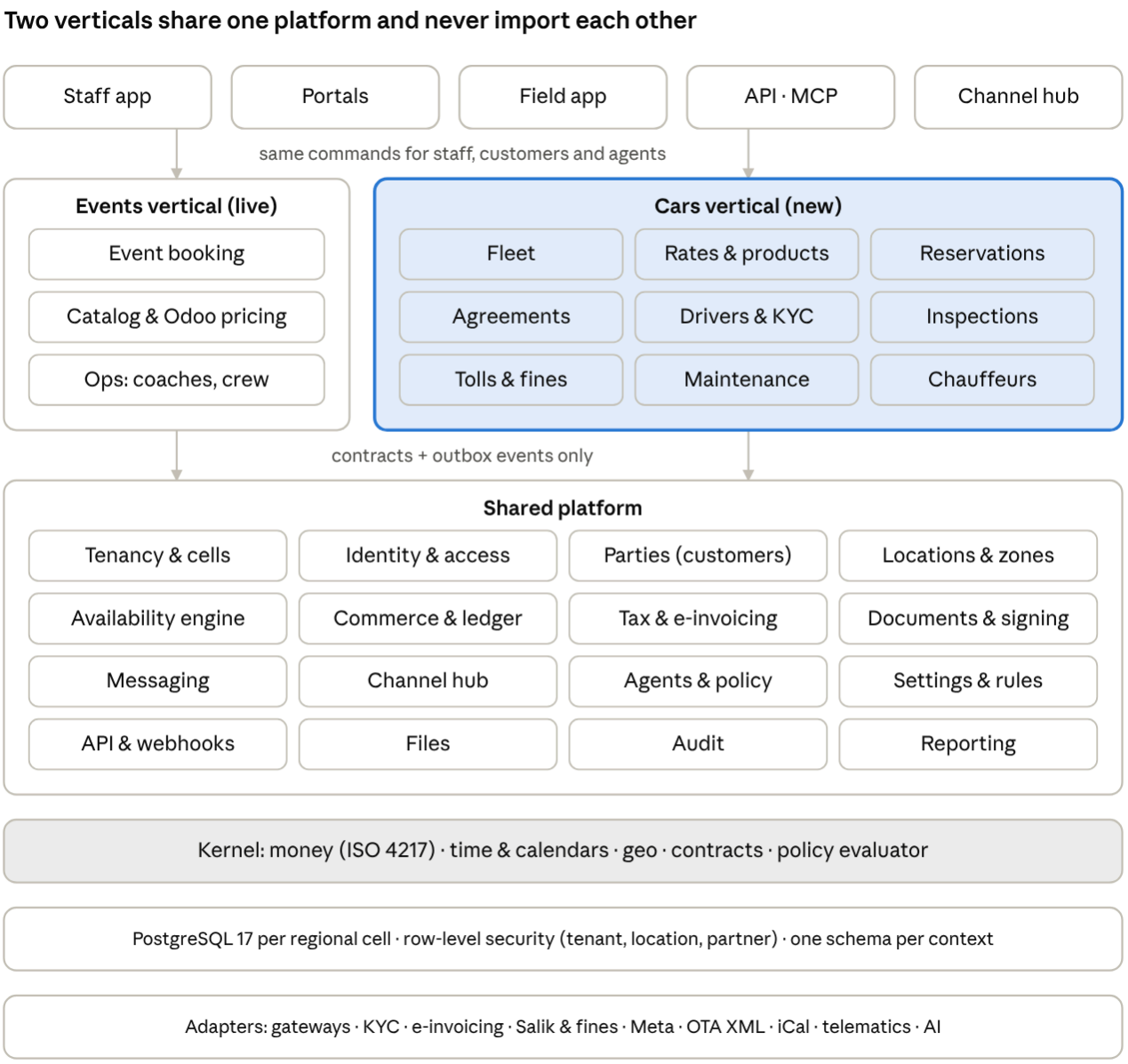
Found in the deeper code read (11 Nov 2026)

#	Finding	Fix
W1	One workspace has one currency and one tax number	Deferred: one company per workspace (section 5)
W2	Card payments have no webhooks; a job searches Stripe every 15 minutes	Webhook ingestion (section 10)

#	Finding	Fix
W3	Each business pastes its own Stripe secret key	Merchant model decision (section 10)
W4	No MFA, passkeys or SSO	Identity hardening (section 13)
W5	Login tables are readable by the app role	Move behind definer functions
W6	One encryption key, no key id, no rotation	Key format v2 + KMS (section 13)
W7	Files pass through the app server; any member can read any private file	Media pipeline (section 12)
W8	Location limits cover only four tables	CI check that every table declares its policies
W9	Lists stop at 200 rows; search is unindexed	Keyset paging and indexed search
W10	Screens refresh by polling every 60 s	Live updates (section 12)
W11	Jobs loop over every workspace	One job per workspace
W12	No error tracking, tracing or alerting	Observability in C0
W13	Migrations run by hand	Pipeline, linter, timeouts
W14	No linter or browser tests in CI	Added in C0
W15	Plans are labels; no billing for the SaaS	Commercial layer (section 10)
W16	The coach portal has no offline support	Offline field app in C5
W17	Messaging is one-way	Inbound handling in C4

4. Architecture: principles, contexts and code structure

The target is a modular monolith. A shared platform sits under two vertical packs (events, cars) that never import each other. Each context exposes only its contract.



target architecture · apps, two verticals, shared platform, kernel, data, adapters

Apps call the same commands for staff, customers and agents. Verticals reach the platform only through contracts and outbox events. Adapters isolate every external system.

Principles

1. **Domain first.** Code is organised by business capability, with one language per context (*Reservation* is not *Agreement*).
2. **High cohesion.** A context owns its tables, rules, state machine and UI fragments.
3. **Low coupling.** Contexts call each other's queries and commands through contracts; changes travel as domain events through a transactional outbox.
4. **Explicit, versioned contracts.** Zod commands, queries and events (`cars.ReservationConfirmed.v1`); a breaking change adds a version.

5. **Pure core, thin shell.** Pricing, eligibility, settlement and policy are pure functions that run in the browser and on the server.
6. **Fast read, strict write.** Reads from projections and caches; writes re-check invariants in the database under a lock or exclusion constraint.
7. **Same rules for humans and agents.** One command path, one permission check, one audit trail.
8. **Extract services only when needed.** Candidates: telematics ingestion, AI vision, e-invoicing.
9. **Region is a tenant attribute.** Data lives in its tenant's cell; the code is the same everywhere.

Contexts

Layer	Contexts
Kernel	Money (ISO 4217 exponents), time and calendars, geo, ids, contract helpers, policy evaluator
Platform	Tenancy and cells · Identity and access (staff, partner, agent principals) · Locations and zones · Parties (customers) · Availability engine · Commerce (orders, invoices, payments, deposits, ledger) · Tax and e-invoicing · Documents and signatures · Messaging (email, SMS, WhatsApp, Instagram, Messenger) · Files · Audit · Integrations (API, webhooks, MCP) · Distribution (channel hub) · Agents · Settings and rules · Billing and entitlements · Risk · Live updates · Search · Reporting · Privacy
Events vertical	Event booking, catalog and Odoo-parity pricing, ops (coaches, crew, surfaces, rain checks)
Cars vertical	Fleet, rates and products, reservations, rental agreements, drivers and KYC, inspections and damage, tolls and fines, maintenance, telematics, chauffeur dispatch

Code layout

- `packages/kernel`, `packages/db-runtime`
- `packages/platform/*` and `packages/verticals/{events,cars}/*`; each context has `contract/ domain/ app/ infra/ ui/ AGENTS.md tests/`

- `packages/adapters/*`: one per external system (payment gateways, KYC, e-invoicing, tolls, fines, Meta, OTA XML, Turo iCal, telematics, AI)
- `apps/web` (staff), `apps/portal` (customers and partners), `apps/field` (inspections, deliveries, chauffeurs), `apps/api` (REST, webhooks, MCP), `apps/worker`, `apps/ingest`, `apps/site`

Boundaries are enforced in CI:

- package `exports` expose only `./contract` and `./ui`;
- dependency-cruiser rules (verticals may use platform contracts, never each other; domain code may not do I/O);
- one Postgres schema per context, with cross-schema foreign keys only to platform identity tables;
- a lint rule against `vertical ===` in platform code.

5. Data model: what is shared and what is separate

Users, customers and a thin order header are shared. Each vertical's booking details live in its own tables. The database stops a car booking from attaching to an event order.

Data	Decision	Why
Users (logins)	One shared table	One person can work for an event and a car business. Splitting breaks sign-in, reset, invites and admin, and gains nothing
Memberships	Per workspace, kind `staff` or `partner`, **a role per vertical**	Events: coach, crew. Cars: counter agent, fleet manager, yard, damage assessor, chauffeur, dispatcher. Partners: adjuster, body shop, referrer, corporate booker
Customers (parties)	One shared core	One card, consent, message history and blacklist flag; one place to export or delete data
Driver profiles, ID documents	Car tables, encrypted	Licence, Emirates ID, QID, passport; own retention

Data	Decision	Why
Orders	Thin shared header `commerce.orders` + `events.bookings` / `cars.reservations` → `cars.rental_agreements`	15 tables already reference `orders`; payments, invoices, tax and e-invoicing are built once

One company per workspace (legal entities deferred, owner decision 13 Nov 2026). A workspace is one company with one currency and one tax registration, as today.

- A business with companies in two countries, such as Dubai and Muscat, uses two workspaces. One login and the workspace picker make switching easy.
- Orders, invoices, payments and ledger rows still store their currency, and company details are read through one function. A legal-entity level can then be added later in one place.
- This is revisited when the first customer asks for two companies in one workspace.

Header contents:

- vertical, number, customer, **bill-to party and payer kind** (customer, company, broker, insurer);
- location, currency, totals, money status;
- a coarse stage (draft, open, confirmed, completed, cancelled) and version.

How the database enforces it:

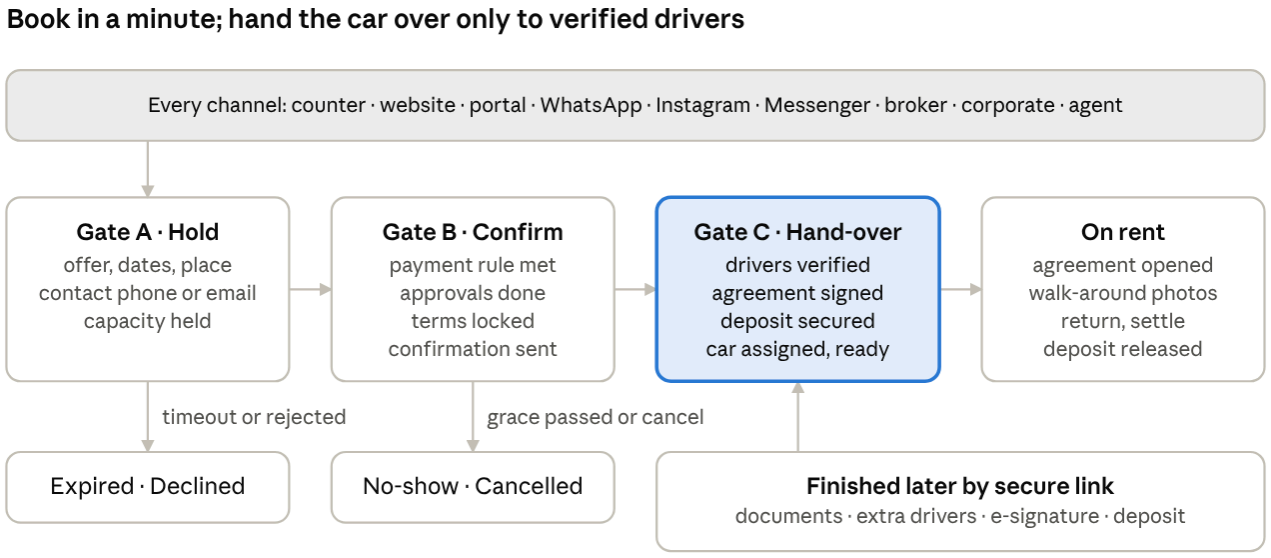
```
create table commerce.orders (
  tenant_id uuid not null, id uuid not null,
  vertical text not null check (vertical in ('events','cars')),
  -- money + stage + bill-to columns
  primary key (tenant_id, id),
  unique (tenant_id, id, vertical)
);
create table cars.reservations (
  tenant_id uuid not null, id uuid not null,
  vertical text not null default 'cars' check (vertical = 'cars'),
  -- car-only columns, NOT NULL where the business says so
  primary key (tenant_id, id),
  foreign key (tenant_id, id, vertical) references commerce.orders
(tenant_id, id, vertical) on delete cascade
);
```

- A deferred constraint trigger checks that every header has exactly one detail row.

- Each vertical owns its detailed status and maps it to the header stage in the same transaction.
- Shared code calls a `VerticalAdapter` (`documentLines`, `messageFields`, `summary`, `onPaymentSettled`, `cancellationPolicy`, `pageUrl`) and never branches on the vertical.
- Money lines are shared (name, quantity, unit, tax, total, payer). Line details are per vertical. JSON is used only for tenant custom fields.
- **Migration:** today's `orders` becomes the header; event-only columns move to `events.bookings` behind a compatibility view; Odoo parity stays green. A 1–2 week spike at the start of C2 proves the header with a car booking before any event data moves.
- **Easy to reverse:** splitting a shared header later is a copy, while merging two order systems later would mean merging money history.

6. Car booking flow

One booking engine serves every channel. A booking passes three gates (Hold, Confirm, Hand-over), each with a checklist computed for that booking. Customers can confirm in about 60 seconds and finish documents and signing later from a link.



car booking gates · hold, confirm, hand-over, on rent

A rule can add a manager approval before Confirm. Items finished later by link move a confirmed booking to Ready, which is the only state that can pass Hand-over.

Risk check. A fraud score is worked out at Hold and again before Confirm. It uses card country against licence country, booking velocity, device, name mismatches, blacklist hits, selfie liveness and the 3-D Secure result. The rules engine turns the score into deny,

approval, a higher deposit or pay-now only. An agent can't confirm a booking that needs review.

People on a booking

Role	Who	Notes
Hirer	Person or company that signs	Always 1
Primary driver	Main driver	Must pass eligibility; often the hirer
Additional drivers	Extra drivers	Each checked; daily fee configurable
Contacts	Assistant, concierge, family	Get updates; can't receive the car unless a setting allows it
Bill-to	Customer, company, broker, insurer	On the shared header
Booker	Staff, corporate booker, broker, agent, hotel partner	Audit and commission
Lead passenger	Chauffeur and transfer modes only	Replaces driver checks

Documents (catalog per country, encrypted, reused until expiry):

- UAE: Emirates ID, UAE licence, passport, visa or entry stamp, home licence, IDP, trade licence for company hirers.
- Qatar: QID, Qatar licence, passport, visa, home licence, IDP.
- Oman: resident card, Oman licence, passport, visa, home licence, IDP.
- US: state licence, passport for international renters, proof of return travel and a second ID for debit-card renters.

Each fact is **declared** (typed) or **verified** (from a document). Search and quotes may use declared facts; **hand-over accepts only verified facts**. Scanning an Emirates ID or licence fills in the form.

What is booked (set per vehicle class):

- **Class**: "Toyota RAV4 or similar", with an upgrade ladder when sold out.
- **Model**: a guaranteed model.
- **Specific vehicle**: common for Dubai luxury and exotic cars.

The optional overbooking allowance is off by default and never applies to specific vehicles.

Where and when:

- pickup and return at a branch, by delivery to an address in a delivery zone, or at the airport with a flight number (a delay extends the no-show grace);
- one-way returns follow a matrix;
- cross-border trips (UAE to Oman) are a product that generates the NOC.

Steps

1. Search (place, dates, optional age and residency)
2. Choose an offer and rate plan
3. Extras and protection (child seats are tracked stock)
4. Renter and drivers (lookup, ID scan prefill)
5. Documents (only what the rules require)
6. Payment and deposit
7. Review and e-sign

Channel defaults (each one configurable)

Channel	Before Confirm	Finished later
Counter / phone	Steps 1–4 and 6	Documents, signing (target ⩽ 90 s for a returning customer)
Website / portal	Steps 1–4 (primary driver basics) and 6	Documents, extra drivers, signing (⩽ 4 screens)
WhatsApp, Instagram, Messenger	Steps 1–4, payment link	Documents via secure link only, never in chat
Broker	Imported as confirmed	Extra drivers, documents, deposit, signing at the counter
Corporate portal	Steps 1–4 (authorised drivers) + PO	Documents if not on file, signing
Monthly	All steps before confirm	—

Payment options:

- pay now in full or in part;
- pay at pickup, with or without a card guarantee;
- broker prepaid;
- corporate invoice;
- cash (counter only).

Deposit strategies: card hold, charge and refund, no deposit with a surcharge, or none. Release after N days for fines and tolls.

Cancellation, no-show and modification policies are versioned objects, locked at confirm. Statuses: Draft → Held → (Pending approval) → Confirmed → Ready → On rent → agreement closed, plus Expired, Declined, Cancelled and No-show.

7. Configuration: settings, rules, guardrails, presets

Everything a business might want different is a setting or a rule, never code. Legal minimums and safety invariants are the only exceptions.

Layered settings. Each context declares its settings once: schema, default, allowed levels, who may edit, and whether the value locks at confirm. The registry generates the Settings screens, validation, API and agent explanations. Levels, from general to specific:

1. platform default
2. country rules (bounds)
3. business
4. branch
5. vehicle class, model, vehicle
6. channel
7. rate plan
8. corporate account
9. one-off override on the booking (needs a permission and a reason)

The most specific level wins, inside the country bounds, and every value shows where it came from ("AED 3,000, from class Luxury, set by Sara").

Rules as data. Each rule is a condition over booking facts, then effects. One pure function evaluates them, with no custom code per business.

- **Facts:** booking, branch, vehicle, customer, each driver, payment.
- **Effects:** deny, require approval, require a document or field (by Confirm or by Hand-over), add a fee, set a value, restrict options, warn, suggest an upsell.
- **Conflicts resolve deterministically:** deny beats approval; requirements combine; fees add up, de-duplicated by code.

Example rules: "visitor whose licence country isn't on the accepted list needs an IDP, passport and visa by hand-over"; "under 25 in a luxury car needs manager approval".

Where rules run:

- at search, to flag or hide offers;

- on every step change, for a live checklist and price;
- at each gate on the server, authoritatively;
- at hand-over, with verified facts only.

Dates and locks.

- Every change is versioned and can be scheduled with an effective date.
- At confirm the booking stores a **policy snapshot**, so rates and cancellation, deposit, mileage, fuel and late-return terms stay as sold.
- Legal rules always apply at their current version.
- Re-pricing on modification is itself a setting: locked terms, current terms, or better of.

Guardrails (never configurable):

- country legal rules can only be tightened;
- availability is checked at commit;
- verified drivers before hand-over;
- signature before hand-over;
- card data never on our servers;
- ID documents never accepted in chat;
- every override audited.

Rule test bench. Describe a sample booking and see every rule that fired, the fees, documents, gates and price. Saved test cases run on every rule change.

Presets (fill settings and rules in one go, all editable): Dubai standard, Dubai luxury, Monthly UAE, Doha, Muscat, US airport, US neighborhood.

Keeping it manageable:

- six setting levels ship first: platform, country, workspace, branch, class, booking. Channel, rate plan and corporate account are added when a customer needs them;
- settings and rules have draft and publish, with a preview of impact ("this change would have affected 37 bookings last month");
- each workspace has a test mode with test bookings, a test gateway and no real messages;
- a workspace's configuration can be exported and imported.

Researched preset defaults

Item	UAE	Qatar	Oman	US
Included distance	250 km/day; 4,500 km/month	Unlimited (capped plans optional)	200 km/day	Unlimited (capped plans optional)
Excess distance	AED 0.75–20/km by class	—	OMR 0.070/km	e.g. $0.25/mi
Deposit	AED 1,000–5,000 by class; release at 21 days, 30-day cap (verify)	QAR 1,500, credit card	OMR 100	$250 credit / $500 debit
Age and fees	By class	21+	21–75; OMR 5/day under 25	State rules; $25/day under 25
Tolls and fines	Salik at cost + AED 1; fine admin AED 50	—	—	Toll pass-through
Late return	29 min grace; hourly to 2 h; extras no grace	same	same	same
VAT	5%, shown inclusive	0% (verify)	5%	Sales tax + surcharges

8. Service modes and roles

Every car booking has a service mode. It changes who drives, who pays, what is checked and how it is priced, through presets and rules on the same engine. Each mode can be switched on per business, branch and channel. Self-drive ships for the pilot; chauffeur, transfers and insurance replacement follow after it.

Mode	Who drives	Who pays	Priced by	Hand-over needs
Self-drive	Customer's drivers	Customer, company or broker	Day, week, month, subscription	Verified drivers, signature, deposit
Chauffeur	Our chauffeur	Customer or company	Hourly with minimum, half or full day, multi-day, monthly	Chauffeur assigned with valid credentials, vehicle ready

Mode	Who drives	Who pays	Priced by	Hand-over needs
Transfer	Our chauffeur	Customer, company or hotel	Zone-to-zone matrix or distance	Same as chauffeur
Insurance replacement	Customer	Insurer (authorised part) + customer (the rest)	Authorised rate × days, plus customer extras	Verified driver, signature, incidentals card (setting)

Chauffeur and transfers:

- Each job needs **two resources**: a car and a free chauffeur. Shifts, rest rules and licence/permit expiry are enforced, and the database refuses overlaps.
- Pricing covers packages, overtime blocks, waiting time from app timestamps, out-of-city surcharges, and fuel and tolls included or passed through.
- Work is assigned on a dispatch board shared with deliveries, which reuses the event crew board and live map.
- The chauffeur app runs en route → arrived → on board → stops → completed, with odometer, expenses and signature.
- Tolls and fines go to the job, not the customer.
- Researched Dubai prices: AED 100–250 per hour, 2–5 hour minimum, full day AED 700–1,500.

Insurance replacement:

1. An insurer account sets rates per class, default days, references, invoice format and payment terms.
2. An adjuster authorises a claim number, rate cap and days (portal, phone or email).
3. The car can be delivered to the body shop.
4. Extensions go to the adjuster through the shared approval inbox.
5. At return the bill splits: the insurer pays authorised days × rate, and the customer pays upgrades, extra days, fuel, tolls and fines.
6. Money owed is tracked per insurer.

Researched UAE norms: 7–15 days, about AED 70–100 per day.

Roles

Role	Kind	Can
Owner, admin, manager, sales, accountant, viewer	Staff	As today, with roles per vertical
Counter agent, fleet manager, yard, damage assessor	Staff (cars)	Bookings, fleet, prep, damage review
Chauffeur	Staff (cars)	Only their own jobs; no prices or customer history
Dispatcher	Staff (cars)	Assign jobs, shifts, live map
Adjuster	Partner (insurer)	Authorise, approve extensions, see their insurer's claims and invoices
Body shop	Partner	Refer customers, update repair dates, see their referrals
Referrer	Partner (hotel, concierge, agency)	Book for guests, see commission
Corporate booker	Partner (company)	Book for authorised drivers, see company bookings and statements
Customer	Portal by link or OTP	Their own bookings, documents, charges

Partner access. Partners use the shared login table with a partner membership. The database adds a **partner scope** (`app.partner_id`) next to today's location scope, so partners only ever see their own records.

Fleet ownership, incidents and cost (added by the review):

- **Ownership:** owned, leased, investor-owned (with an owner's revenue share and statement) or hired in from another operator (with a supplier payable). Hired-in cars add supply for their period.
- **Incidents and claims:** accident, theft or impound, with police report, claim on the fleet's insurance, repair and time off the road.
- **Cost and lifecycle:** purchase or lease cost, depreciation and disposal, so reports can show profit per vehicle.
- **Government contract registration:** an adapter for countries that require rental contracts to be registered (to be verified per country).

9. Channel hub: Meta, marketplaces, brokers, P2P, Google

Every channel feeds the same booking engine, with the same availability, rules, money and audit. Each business configures each channel separately. The hub is a platform context, so events can use it too.

Channel	Examples	How bookings arrive	Notes from research
Meta messaging	WhatsApp, Instagram DMs, Messenger, click-to-WhatsApp ads	One inbox for staff and the AI agent; WhatsApp Flows for in-chat forms; car lists; payment links; Conversions API events	Dubai operators often sell on Instagram and close on WhatsApp
Lead marketplaces	OneClickDrive	Leads by call, WhatsApp or email	Flat monthly subscription, no commission
Brokers and aggregators	Rentalcars.com, Discover Cars, EconomyBookings, VIPCars, QEEQ	Voucher bookings, prepaid or pay at desk	The supplier holds the contract and deposit
B2B distribution	CarTrawler	OpenTravel XML: availability and rates queried live, then booked	2,200+ suppliers
P2P (US)	Turo, Getaround	iCal sync + booking-email parsing	No public host API found; current tools all work this way
Search	Google Business Profile, Google Ads	Booking links per branch, schema.org on booking pages	Google's vehicle ads are for cars for sale, not rentals
Partners	Hotels, concierges, insurers, corporates	Partner portal	Commission statements
GDS (later)	Sabre, Amadeus, Travelport	Corporate travel	Only after GA

Per channel, each business configures:

- **Scope:** branches, classes or vehicles, service modes.
- **Content:** photos and descriptions per language.

- **Rates:** same as direct, ± %, own rate plan, or net rate, with a rate-parity guard.
- **Inventory:** allotment per class per day, safety buffer, stop-sell, close-out dates, lead time, minimum and maximum length.
- **Policies:** cancellation, deposit, payment model, required documents.
- **Acceptance:** auto-confirm, request-to-book within N minutes, or auto-decline rules.
- **Money:** commission, who invoices, payout import.
- **Customer ownership:** marketing to broker customers off by default.
- **Bot and hand-off rules, tracking, and a kill switch.**

Reliability:

- each inbound booking is created once, keyed by the channel's booking id;
- outbound pushes go through the outbox, throttled;
- live queries are answered from the availability projection, then checked under lock;
- a conflict queue catches oversells;
- reconciliation runs daily, and each channel has a health view.

Build order:

1. Meta messaging
2. OneClickDrive leads + Turo/Getaround sync
3. OpenTravel XML endpoint, certified with one broker
4. Google links and structured data
5. GDS later

For the pilot, only the WhatsApp inbox (staffed by people) and the operator's own website ship. The other channels follow in a wave after the pilot.

10. Money, payments, tax, tolls and fines

Money is built once on the shared commerce context and works in any currency, any gateway and any tax system the two markets need.

Money. Amounts are stored in minor units with a currency. The decimal count comes from ISO 4217 (USD, AED, QAR 2; OMR, KWD, BHD 3). Rounding lives in one place and follows the tax jurisdiction. Legacy `*_cents` columns stay for USD event tenants, and a lint rule blocks ad-hoc `/100` maths.

Payment gateway port. Every gateway implements authorize, increment, capture, void, refund and merchant-initiated charges, and declares its capabilities (holds, incremental auth, maximum hold days, 3DS).

- **Stripe:** US and UAE.

- **GCC shortlist:** Checkout.com, Network International (N-Genius), PayTabs, Telr, Amazon Payment Services, plus a local acquirer for Qatar. Stripe doesn't onboard Qatar businesses.
- Card data never touches our servers (hosted fields).
- Cash and bank transfer stay first-class; BNPL (Tabby, Tamara) comes later.

Who the merchant is. The gateway port supports two models, chosen per workspace:

- **Own gateway:** the business's own account and keys. This is today's model and the pilot's.
- **Platform:** Stripe Connect or a payment-facilitator partner, where Rent Hatch can take a fee per transaction. It is evaluated before GA.

Webhooks and disputes (missing today):

- gateway events arrive through a signed webhook endpoint with a timestamp check, an event store and replay; handlers are safe to run twice. Polling stays as the safety net;
- card disputes (chargebacks) are a workflow. The evidence pack is built from the signed agreement, the ID check, condition reports, photos and toll or fine statement lines;
- hold expiry and payouts are tracked from the same events.

Ledger. Each order has double-entry-lite accounts:

- revenue and VAT payable;
- deposit liability;
- receivables from customers, companies, brokers and insurers;
- pass-through fines.

The ledger is reconciled with the gateway every 15 minutes.

Tax and invoices:

- A tax engine works out components per branch jurisdiction: US sales tax plus rental surcharges (Avalara optional); UAE and Oman VAT 5%; Qatar none today (verify).
- **Issued invoices never change;** corrections are credit or debit notes.
- Invoices are bilingual (EN/AR), carry the TRN and the words "Tax Invoice", and can bill to a customer, company, broker or insurer.
- **E-invoicing adapters:**
  - UAE PINT AE through an accredited provider: mandatory from 1 Jan 2027 for revenue ≥ AED 50m, and from 1 Jul 2027 for others.
  - Oman Fawtara: phased from Aug 2026, with dates being revised.

Tolls and fines (post-rental charges):

1. **Import** Salik/Darb statements, police and RTA fines (Dubai, Sharjah, Abu Dhabi), Qatar MOI and Oman ROP fines, and US toll files. Imports come first; an API only where one is officially confirmed.
2. **Match** each line to the rental by plate and time. Unmatched lines go to an exceptions queue.
3. **Price** each line at cost plus the business's admin fee, with VAT on Salik per current rules.
4. **Collect** from the retained deposit or the saved card, under consent given in the signed agreement.
5. **Notify** the customer with evidence (gate, time, plate). Disputes have their own workflow.
6. **Release** the deposit on schedule, never later than the jurisdiction cap (30 days in Dubai, reported; verify).

This directly fixes the top complaints found in research: deposits held too long, and Salik and fines charged without proof.

Billing for Rent Hatch itself. Plans are only labels today. A platform billing context adds:

- **entitlements:** features and limits by plan, checked in one place;
- **metering:** usage events for active vehicles, bookings, messages, ID checks, agent runs and e-invoices;
- **subscriptions:** priced per active vehicle, with trials, invoices in the right currency and tax, and failed-payment handling;
- **prepaid usage balances** for pass-through costs (WhatsApp, SMS, ID checks, AI). Agent budgets draw on them, so a busy workspace can't run up unbounded cost.

Accounting. Revenue from monthly rentals is spread across the period. Deposits are a liability. Receivables are aged by payer. There is a general-ledger export per workspace.

11. Agentic architecture

Agents act only through the same commands as staff, checked against a per-business policy. Anything that moves money, claims damage or cancels a confirmed rental needs a person's approval until the business chooses otherwise, within platform limits.

Agents and their limits

Agent	Channel	Default autonomy	Hard limits
Ops copilot	Staff app	Reads; drafts only	Read-only tools plus drafts
Document extraction	Portal, counter	Proposes fields	A person or the KYC vendor confirms
Reservation agent	Web chat, then WhatsApp, Instagram, Messenger	Acts with approval, then within limits	Discount ⩽ business cap; no cancelling confirmed rentals; documents only via secure link
Readiness / KYC chaser	WhatsApp, email, portal	Within limits for chasing	Never approves a failed check
Fines and tolls matcher	Back office	Acts with approval; tolls under a cap within limits	Disputes and large fines go to a person
Damage assistant	Field app	Suggest only	No charge without a person confirming; evidence shared with the customer
Fleet and yield advisor	Back office	Proposals	Overdue swaps, repositioning, overbooking and price suggestions
Collections	Email, SMS, WhatsApp	Acts with approval	Follows the dunning policy

Controls and boundaries:

1. **Identity.** Each agent is a principal per business, with a role, a subset of permissions, a location scope and an autonomy level: suggest, act with approval, act within limits, or autonomous. Every action records the agent, on whose behalf, and who approved.
2. **Tools come from contracts.** Command metadata generates each tool's schema, risk class and permission, so no hand-written tool can bypass a handler.
   - Risk classes: read, reversible write, irreversible, money movement, customer messages, PII access.
3. **Policy engine.** Per-business rules (e.g. "money ⩽ AED 500 allowed, above needs approval") sit under platform guardrails the business can't loosen.

4. **Dry run first.** Commands return the quote or diff before writing. The approvals inbox shows the diff, the reason and the evidence. It is the same inbox used for human rule approvals.
5. **Subject binding.** A customer conversation is bound to a verified customer (OTP), and its tools only touch that customer's bookings. There are no list or export tools.
6. **Untrusted input.** Customer messages, uploads and statement files are data, never instructions.
7. **Data minimisation.** No card data reaches a model; ID numbers are masked; ID images go only to in-region models where residency applies.
8. **Limits.** Rate limits, daily cost budgets, maximum steps per run, and a kill switch per agent and per business (plus platform-wide).
9. **Observability.** Every run and step is logged with model, prompt version, tool, cost and outcome. Traces join agent → tool → command → SQL.
10. **Evals and rollout.** Each agent has golden, adversarial, prompt-injection and EN/AR test conversations in CI. Agents run in shadow mode first, and autonomy rises per business on measured precision.
11. **Models.** Routing per cell: claude-haiku-5-5 for classification and matching, claude-sonnet-5-5 for conversations, claude-opus-5-5 for complex back-office reasoning.
12. **MCP server.** Exposes the same governed tools, with scoped keys, to a business's own AI assistants.

Coding agents in the repo:

- each context has an `AGENTS.md` with its invariants;
- contract tests catch broken consumers;
- the dependency-cruiser gate blocks shortcuts between contexts.

Added by the review:

- **Model location.** Each agent feature declares the data it sends. A workspace with residency switched on runs only features whose model endpoint is in its region. In-region availability must be confirmed before agents handle Gulf personal data.
- **Knowledge store.** Policy answers come from a per-workspace store in the cell's database, under the same row-level security.
- **Durable runs.** Conversation and process state is saved in the database and resumed by jobs, so a multi-day chase survives restarts.
- **Cost and fraud.** Every run is metered against the workspace's balance, and the risk check runs before an agent confirms a booking.

Agent boundaries: each agent does its job and nothing else

Every agent has a written job charter. An attempt to go beyond it is blocked before it has any effect, the run is terminated at once, and a person takes over. The limits are enforced by code outside the model, because a model can be tricked or can make mistakes. The full plan is in `docs/car-rental/agent-boundaries.md`.

The job charter (versioned data, approved by a person; the agent can't read or change it)

Field	Example: reservation agent
Purpose	Quote, book and change self-drive rentals for one verified customer
Tools allowed	Search offers, hold, update, confirm, readiness, payment link, upload link
Data scope	Only the bound customer's bookings, in its workspace and branches
Limits	Discount cap, money per action, actions per conversation, steps per run, cost per day
Needs approval	Cancelling a confirmed rental, waiving a fee
Never	Refunds, damage charges, ID documents, settings, other customers, exports
Topics	The workspace's cars, prices, policies and the customer's own bookings
Hand-off	The branch's staff inbox

A workspace can narrow a charter but never widen it beyond the platform's version.

Five walls outside the model:

1. **Tool list.** The agent is only shown the tools in its charter.
2. **Run token.** Each run gets a short-lived credential bound to one agent, one workspace, one customer and the charter's tools.
3. **Policy check.** Every call is checked against the charter's limits before the command runs.
4. **Database scope.** Row-level security refuses other records even if every earlier wall failed.
5. **Output check.** Every reply is checked before sending: on-topic, no promises outside policy, no one else's data.

Agents also can't change their own charter, prompt or permissions, and can't start or instruct another agent. A few fake "canary" tools that no job needs exist only to catch misuse.

What counts as going beyond the job

Severity	Examples
S1	A forbidden or canary tool; another customer's, branch's or workspace's data; trying to change its own permissions; reading ID or payment data it doesn't need
S2	Exceeding a discount, amount, step or cost limit; bypassing an approval or retrying a denied action another way
S3	Advice or promises outside its topics

An action that is in the charter but needs approval isn't a violation. It goes to the approvals inbox.

What happens, always and in the same request:

1. The action is blocked, so nothing changes.
2. The run is terminated and its token revoked.
3. The conversation goes to a person, with the transcript.
4. A violation record is written to the audit trail.
5. The workspace's managers are alerted; S1 also alerts the platform team.

Then, by severity:

- **S1:** the agent is suspended for that workspace until a person reviews it. If the same agent version shows S1 in several workspaces, that version is switched off everywhere.
- **S2:** the agent is suspended for that customer. Three S2 events in 24 hours suspend it for the workspace.
- **S3:** the reply is replaced with a safe one. Repeated S3 events lower the agent's autonomy by one step.

Suspension is scoped on purpose. A customer can type instructions into a chat to provoke a violation, and one such message must not switch the agent off for everyone. A suspended agent comes back only when a person has read the record and reinstated it, with a narrower charter if needed.

Keeping agents inside their job: a boundary test suite must pass before each release (zero breaches reaching a command); new versions run in shadow mode first; dashboards track blocked calls and terminations; charters are re-approved each quarter. The work is steps B1–B8 of track C8, weeks 6–22.

12. Regional cells, scale, speed and security

The platform runs as identical cells, one per region. A tiny global directory maps each workspace to its cell. New cells are added as load grows.

Cells:

- **Control plane (global):** workspace directory (slug → cell), platform billing, platform admins, marketing site, and the central login store (with a standby). No customer PII.
- **us-west:** today's Vercel (pdx1) + Supabase (Oregon) + Railway.
- **me-central (UAE):** AWS me-central-1, Azure UAE North, or Supabase/Vercel if they offer a UAE region (decision D2). It holds a PG17 primary + replica, pooler, in-region object storage, KMS, worker and ingest.
- Qatar and Oman workspaces start in the UAE cell unless law requires in-country hosting. Residency is a workspace setting, subject to legal review of UAE PDPL Arts. 22–23, Qatar PDPPL and Oman PDPL.
- Routing: the edge resolves the workspace's cell and serves it from that cell's hostname. There is one login for all regions (owner decision 13 Nov 2026): accounts and sessions live in a central login store that both cells trust. Staff names, emails and password hashes are therefore held centrally, outside the UAE cell, while customer, driver and booking data stays in its cell. This goes in the privacy terms and the legal review. Tenant moves reuse the existing `copy-tenant.ts`.

Scale inside a cell:

- transaction pooler (already `prepare:false`);
- read replica for reports and agents;
- named worker queues (critical, messaging, projections, ingest, reports, ai) with per-tenant fairness;
- monthly partitions with retention for telematics, messages, audit and post-rental charges;
- telematics through a separate ingest service that only emits meaningful domain events.

Added by the review:

- **Live updates:** outbox events are published to Redis and pushed to open screens over a stream. The fleet calendar, yard board, dispatch, approvals and chat inbox use it.

- **One job per workspace:** a schedule table replaces the loops over every workspace, so one slow workspace delays only itself.
- **Search and paging:** keyset paging on every list, and indexed search for names, phones, plates, VINs and document numbers.
- **Photos and documents:** direct uploads to an in-region bucket with signed URLs, access by file type, thumbnails and malware scanning. Each photo is hashed and time-stamped, and condition reports are chained so a later change is detectable.
- **Reporting:** a reporting schema fed by events and read from the replica, with a daily fleet snapshot for utilization.
- **Release safety:** migrations follow expand, backfill, switch, contract, and are applied by the pipeline one cell at a time.
- **Recovery:** point-in-time recovery and a quarterly restore drill per cell. Targets: restore within 4 hours, lose at most 15 minutes of data (to be confirmed).
- **Privacy operations:** every context can export and erase one person's data, with legal hold. A register lists each outside service that touches personal data, per cell.
- **Booking websites** on the operator's own domain, routed at the edge, with cached pages that search engines can index.

Speed budgets

Surface	Budget
Booking widget availability + quote	p95 ⩽ 400 ms at 1,000 vehicles per tenant
Reservation commit (lock + insert + outbox)	p95 ⩽ 250 ms
Live quote in the browser	< 200 ms
Staff list pages	p95 ⩽ 300 ms (existing budget)
Fleet calendar, 500 vehicles × 14 days	First render ⩽ 1.5 s, virtualised
Check-out on a phone	⩽ 2 min, works offline
Page response from Dubai	p95 ⩽ 300 ms

Security and privacy:

- row-level security tenancy as today, plus location and partner scopes;
- field-level encryption (per-tenant keys, KMS per cell) for licence numbers, ID numbers and dates of birth;
- ID images in private storage with short signed URLs and every view logged; purged after closure unless disputed;

- retention settings for US DPPA and state laws, UAE PDPL, Qatar PDPPL and Oman PDPL;
- PCI scope kept minimal with hosted fields; signed, replay-protected webhooks; read-only support impersonation (exists);
- blacklists scoped to each business by default.

Language and locale:

- ICU messages in en-US, en-AE, ar-AE, ar-QA and ar-OM;
- RTL with logical CSS properties, starting with the component kit;
- weekend days per country (UAE Sat–Sun; Qatar and Oman Fri–Sat) and seasons such as Ramadan, Eid and summer, all in branch calendars;
- WhatsApp Business as a messaging channel.

13. Security architecture

Security is layered: an edge firewall, limits in the app, limits in the database, isolated background work, and monitoring. No single control is enough. The full design is in `docs/car-rental/security.md`.

Where the product stands today

In place	Missing
Tenant isolation in the database	A firewall and rate limits in front of the app
Sign-in never reveals whether an account exists	MFA, passkeys, SSO
Rate limit on sign-in routes	Rate limits on everything else
Clickjacking and content-type headers	A full Content-Security-Policy and HSTS
Card numbers never reach our servers	Payment webhooks with signature checks
Secrets encrypted at rest	Key rotation, a KMS, keys per workspace
Turnstile and limits on marketing-site forms	The same on app sign-up, reset, booking and one-time codes
Nightly backups	Tested restores, error tracking, alerting

The request-flood case. Someone loops requests at a page that doesn't exist, such as `renthatch.org/access`.

- The marketing site is a static site on shared hosting with nothing in front of it, so a flood can exhaust the host.
- The app runs on functions that scale out, so one processor doesn't choke. The real risks are used-up database connections and a growing bill.

Layer	Control	Effect on the flood
Edge network	CDN and firewall in front of both sites, with always-on DDoS protection	Large floods are absorbed before they reach us
Edge rate limit	Limits per IP address and network; a challenge, then a block, for repeat offenders	The loop is cut off within seconds
Cached "not found"	404s served from the edge; the path is checked against the workspace-address pattern before any database call	A request for `/access` costs almost nothing
Bot challenge	Turnstile on forms; a managed challenge for automated traffic	Scripts fail; people don't notice
App limits	Limits per user, workspace and API key, counted in Redis; body size caps; request timeouts	One account can't starve the others
Database limits	Statement and idle timeouts, a pooler with a queue limit, a read replica	A slow query is killed, not queued forever
Work isolation	Job queues by priority; one job per workspace	An import flood can't delay payments
Load shedding	`429 Too Many Requests` with a retry time; circuit breakers on outside services	The system slows down in a controlled way
Cost guard	Spend alerts and caps on hosting, SMS, WhatsApp, ID checks and AI	An attack can't become an unbounded bill
Detection	Alerts on request rate, error rate, pool saturation and queue depth	We know within minutes

Load balancing. In the US, Vercel already balances and scales the app; the worker should run at least two instances. The UAE cell needs a load balancer in front of at least two app instances in two zones, with health checks, auto-scaling and a database standby.

Redis (managed, one per region, short-lived data only):

- rate-limit counters;
- cache for availability, resolved settings and sessions;
- idempotency keys and one-time-code attempts;
- publish/subscribe for live updates.

Every cache key includes the workspace id.

Bots, crawlers and scrapers:

- Turnstile (reCAPTCHA is an equivalent) on sign-up, password reset, booking, quote-page actions, and on sign-in after failed attempts;
- a challenge and limits per phone, IP and country before any SMS or WhatsApp code is sent, which blocks paid-message fraud;
- `robots.txt` and `noindex` on the app, portals and tokenised pages; search crawlers allowed on booking websites; a stated policy for AI crawlers;
- availability queries rate-limited and tied to a short-lived signed token, to slow price scraping;
- limits per account and per IP, a breached-password check, and alerts on sign-in from a new country.

Accounts and phishing:

- MFA for roles that move money, see ID documents, change settings or manage people; passkeys preferred, and required for platform admins;
- step-up confirmation for refunds, exports, viewing ID documents and changing bank or gateway details;
- a device list, sign-out everywhere, and sessions ended on password change;
- SPF, DKIM and DMARC at reject for our domain; businesses verify their own sending domains;
- payment, upload and signing links only on our domain or a verified business domain, single-purpose and expiring;
- sending limits for new workspaces, link scanning in templates, an abuse-report address and fast suspension, so the platform can't be used for phishing;
- lookalike workspace names blocked; security notices for new sign-ins, changed bank details and new API keys.

Application and API security:

- a full Content-Security-Policy with nonces; strict input schemas with length caps;
- uploads direct to the bucket with size, type and image-dimension limits and malware scanning;

- outbound calls to addresses that businesses type in (mail servers, calendar feeds, webhooks) are checked and routed through an egress allow-list;
- OAuth with scopes; API keys that are hashed, scoped, expiring and rotatable;
- quotas per key and workspace, page-size caps and idempotency keys on every write;
- incoming webhooks checked for signature and timestamp; outgoing webhooks signed, with a delivery log;
- the broker XML endpoint parses with external entities switched off and strict size limits;
- a connected-car command such as "unlock" is allowed only during an active rental and is always logged;
- dependency, secret and static scanning on every pull request.

Data protection:

- encryption in transit and at rest; field-level encryption for licence and ID numbers with per-workspace keys in a KMS;
- ID documents in private storage, behind links that expire in minutes, with every view logged;
- backups encrypted, kept in-region and restore-tested each quarter;
- no personal data or secrets in logs; staff access to production data is time-limited and logged.

Detection and response:

- alerts on failed sign-ins, permission denials, mass exports, new API keys and changed payout details;
- an incident response plan with the notification deadlines of each country (to be verified);
- a `security.txt` file and a disclosure policy;
- independent penetration tests before the pilot, before GA, and then yearly.

Compliance

Area	What applies
Cards	PCI DSS at its lightest level (SAQ A), because the gateway hosts card entry
Company assurance	SOC 2 Type II, then ISO 27001; insurers, brokers and corporates will ask

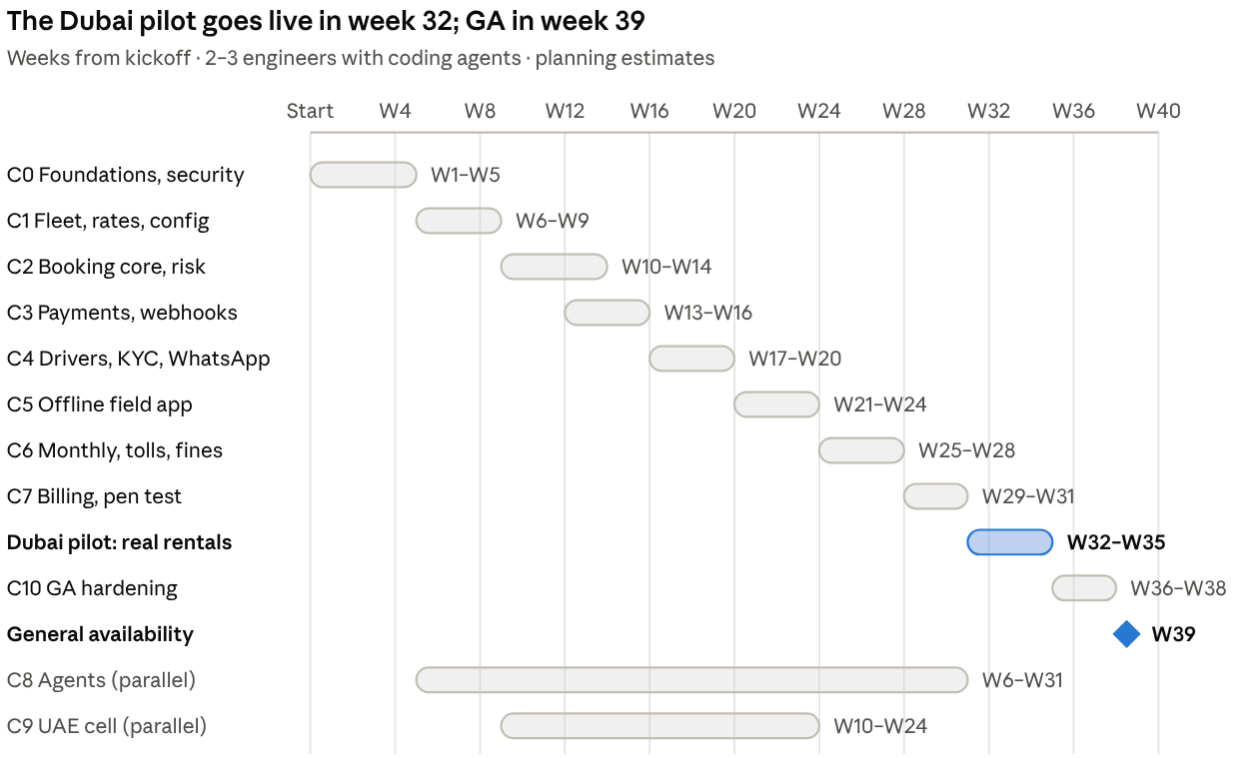
Area	What applies
Privacy	US state laws and the Driver's Privacy Protection Act; UAE PDPL, Qatar PDPPL, Oman PDPL; GDPR may apply to European tourists (verify)
Messaging	US TCPA and CAN-SPAM; Meta's WhatsApp Business policies
Signatures	US ESIGN and UETA; Gulf electronic transactions laws (verify)
Tax	UAE and Oman VAT invoices and e-invoicing
Vendors and people	Agreements and reviews for every outside service that touches personal data; company SSO with MFA, managed devices, training

Priority order

1. **Now, in C0:** firewall and rate limits in front of both sites; database timeouts; error tracking and alerts; MFA for owners and admins; full Content-Security-Policy and HSTS; scanning in CI; DMARC at reject.
2. **Before the pilot:** Redis limits; Turnstile on app forms and one-time codes; payment webhooks; KMS and field-level encryption; direct uploads with scanning; egress controls; step-up and passkeys; a tested restore; the first penetration test; an incident plan.
3. **Before GA:** OAuth, quotas and webhook signing complete; SSO; central alerting; SOC 2 audit period started; a second penetration test; the vendor register per region.

14. Development plan

A Dubai operator goes live around week 32 with self-drive daily and monthly rentals, runs real rentals for 4 weeks, and GA follows around week 39. The US launch, chauffeur, insurance replacement, brokers and most agents follow in waves from week 40. Agents and the UAE cell run as parallel tracks. Each phase ships on its own branch (`cars/phase-cN`) through a Vercel preview, owner QA and a PR into `main` . Events stays green on every merge: e2e scripts, DB tests, and Odoo parity at 150/150.



car rental roadmap · weeks from kickoff

C3 overlaps the end of C2. The UAE cell must be live by week 24, before the field app is tested in Dubai. This is revision 3 of the plan: the pilot is narrower than before, and the work found by the architecture review comes before it.

1. **C0 · Foundations (≈ 5 weeks, weeks 1–5)**
   - Scope: kernel (Money, calendars, contract helpers); **currency stored on every money row; settings registry with six** levels, rules evaluator, draft and publish; approvals inbox; shared order header; outbox and one job per workspace; **MFA and step-up**; key format v2; linter, browser tests, migration rules and the table-policy check in CI; **security baseline** (firewall, edge rate limits, cached 404s, database timeouts, CSP, scanning, DMARC); observability; i18n and RTL base.
   - Exit: events fully green; a request loop at an unknown path is cut off at the edge and causes no database calls; existing workspaces work unchanged.
2. **C1 · Fleet, catalog, rates, policies, presets (≈ 4 weeks, weeks 6–9)**
   - Scope: classes, models, vehicles with **ownership and cost**; branches and delivery zones; extras and protection; pure rate engine incl. monthly; policies; Dubai presets; Settings UI, rule test bench, impact preview and **test mode**; paging and search base.
   - Exit: 30+ golden quotes in AED plus OMR 3-decimal cases; reference cases R1–R8 pass.
3. **C2 · Booking core (≈ 5 weeks, weeks 10–14)**

- Scope: shared-header spike first; availability by class, model and vehicle; reservation with parties, gates and readiness; **risk checks**; counter builder; fleet calendar; **live updates**; one search box.
- Exit: 50 parallel holds for the last car → exactly 1 succeeds; a high-risk booking can't be confirmed without approval; counter booking ⩽ 90 s; a change shows on another screen within 2 s.

4. **C3 · Payments, webhooks, disputes, ledger, tax invoices (≈ 4 weeks, weeks 13–16)**
   - Scope: gateway port; first GCC gateway; **webhook ingestion**; **dispute evidence packs**; pay now, pay at pickup with guarantee, cash; deposit strategies; ledger; immutable bilingual invoices.
   - Exit: the AED lifecycle reconciles to the minor unit; a replayed webhook changes nothing; issued invoices can't change.
5. **C4 · Drivers, documents, KYC, media, customer channels (≈ 4 weeks, weeks 17–20)**
   - Scope: driver profiles with KMS field encryption; **media pipeline**; declared vs verified facts; KYC vendor; agreement e-sign; website widget with Turnstile; "complete your booking" portal; two-way messaging; WhatsApp inbox staffed by people; Redis limits and egress controls.
   - Exit: unverified drivers can't pass hand-over; web → portal → ready works in EN and AR; every ID document view is in the audit trail.
6. **C5 · Offline field app: hand-over, return, delivery (≈ 4 weeks, weeks 21–24)**
   - Scope: installable offline app (new work); check-out and check-in with photos and damage diagram; **evidence integrity**; delivery and collection tasks; settlement; extensions and swaps; yard board.
   - Exit: check-out ⩽ 2 min with the network off; a changed photo or report is detected; automatic deposit release.
7. **C6 · Monthly, tolls, fines, maintenance, incidents (≈ 4 weeks, weeks 25–28)**
   - Scope: recurring billing and swaps; Salik, Darb and fine imports, matching, collection with evidence, disputes; maintenance; **incidents and fleet insurance claims**; **investor-owned and hired-in cars**; reporting schema and daily fleet snapshot.
   - Exit: three monthly cycles reconcile; ⩾ 95% of toll lines auto-match with no double charges; a correct owner statement.
8. **C7 · SaaS billing, security hardening, pilot readiness (≈ 3 weeks, weeks 29–31)**
   - Scope: **entitlements, metering, subscriptions, prepaid usage balances**; privacy export and erase; tested restore; incident plan; first penetration test; pilot onboarding in test mode.
   - Exit: plan limits enforced in one place; restore meets the recovery targets; no open high-severity finding.

9. **Pilot · One Dubai operator (weeks 32–35)**
    - Scope: real rentals on the UAE cell; daily review of exceptions; the pilot's past bookings become pricing golden cases.
    - Exit: the success metrics are met or have a dated fix.
10. **C10 · GA hardening (weeks 36–38), GA in week 39**
    - Scope: pilot fixes; booking websites on the operator's own domain; public API docs, OAuth apps and scoped keys; accounting exports and importers; 10× load test; second penetration test; SOC 2 audit period started.
11. **C8 · Agents (parallel, weeks 6–31)**
    - Scope: agent platform, knowledge store and per-cell model routing; ops copilot, document extraction and the reservation agent (web chat, then the WhatsApp inbox) before the pilot.
    - Exit: zero policy violations in adversarial evals; ⩾ 95% correct bookings in eval conversations.
12. **C9 · UAE cell, residency, e-invoicing (parallel, weeks 10–24)**
    - Scope: the cell as infrastructure-as-code with a load balancer, two zones, database standby, Redis and KMS; directory routing and the central login store; legal review; UAE PINT AE adapter.
    - Exit: Dubai data, backups and logs stay in region; a restore drill passes.

After the pilot (from week 40, order to be confirmed)

Wave	Scope	Size
P1 · US launch	US state packs, airport fees, debit-card rules, US tolls, Turo and Getaround sync, a US pilot	≈ 5 weeks
P2 · Chauffeur and transfers	Chauffeur resource, packages, dispatch, chauffeur app	≈ 4 weeks
P3 · Corporate and insurance replacement	Corporate accounts, insurer accounts, split billing, partner portal, SSO	≈ 5 weeks
P4 · Channel hub	Brokers by voucher then XML, OneClickDrive, Instagram and Messenger, Google	≈ 5 weeks
P5 · Qatar and Oman	Country packs, local gateway, Oman e-invoicing, cross-border product	≈ 4 weeks
P6 · More agents	Chaser, fines matcher, damage assistant, yield advisor, collections	≈ 5 weeks

Wave	Scope	Size
P7 · Later	Telematics and keyless pickup, native app wrappers, platform merchant model, GDS, BNPL	—

15. Testing, success metrics and risks

Every money path has golden cases, every race has a concurrency test, and every agent change passes an adversarial eval before merge.

Tests

Layer	What
Domain unit	Rate engine, settlement, rules evaluator, settings resolver, state machines, readiness
Property	Rule conflicts resolve deterministically; locked rules can't be loosened; snapshot hashes are stable
Golden and reference	16 reference cases now (R1–R16); real past bookings priced to the minor unit once pilots onboard
Contract	Command and event schemas per version; consumers checked against producers
Database	RLS as the app role; tenant, location and partner scopes; exclusion constraints; lock races; header ↔ detail integrity; invoice immutability
End to end	Counter, web, portal, field app; desktop and phone; Arabic RTL; timing checks
Agent evals	Golden, adversarial, prompt-injection, EN/AR; zero policy violations
Performance	Search + quote, commit, offline field app, Dubai latency

Success metrics

Metric	Target
Counter booking, returning verified customer	⩽ 90 s
Counter booking, new customer with ID scan	⩽ 3 min
Website screens to confirm	⩽ 4
Bookings ready before pickup day	⩾ 70%
Hand-over for a ready booking	⩽ 2 min
Tolls auto-matched	⩾ 95%
Deposits released within the jurisdiction cap	100%
Agent policy violations	0

Risks

Risk	Mitigation
The shared order header grows fat	Header-column rule + schema check in CI; C2 spike proves the boundary
Shared code starts branching on the vertical	Adapter interface + lint rule
GCC gateway can't onboard Qatar/Oman or hold deposits	Gateway port; confirm with providers before C3; charge-and-refund fallback
Legal rules (licences, deposit cap, e-invoicing dates) are wrong or change	Country packs as versioned data marked "verify"; legal review in C9
Salik and fines have no official API	Statement imports first; API adapter only when confirmed
Turo and Getaround have no public API	iCal + email parsing, as current tools do
Agents make costly mistakes	Approvals, budgets, kill switches, shadow mode, evals
Event regressions during refactor	Strangler migration; parity and e2e on every merge
No real pilot data yet	Reference cases now; real golden cases at pilot onboarding

16. Open decisions and sources

Thirteen decisions remain. D16 is needed in C0, D2 by week 10, and D1 and D4 before C3.

#	Decision	Recommendation	Needed by
D1	GCC payment gateway	The one that onboards UAE, Qatar and Oman businesses with deposit holds	C3
D2	UAE cell hosting	Confirm provider regions; otherwise AWS me-central-1	Week 10
D3	Qatar/Oman data residency	UAE cell, with legal sign-off	Wave P5
D4	Default deposit strategy	Charge-and-refund in GCC, card hold in US	C3
D5	KYC vendor	One global vendor; UAE Pass later	C4
D6	Document and licence lists per country	Owner verifies; researched lists ship marked "verify"	Pilot
D7	The Dubai pilot operator	One operator with 50–100 past bookings to share	C6
D11	Merchant model	Own gateway for the pilot; platform model evaluated before GA	C10
D12	SaaS pricing	Per active vehicle, plus prepaid usage for pass-through costs	C7
D13	Native apps	Installable web app first; wrap for app stores if iPhone limits bite	After the pilot
D14	AI model region for Gulf data	Confirm in-region availability	Before agents handle personal data
D15	Government contract registration	Verify what the UAE, Qatar and Oman require	C6
D16	Recovery targets	Restore within 4 hours; lose at most 15 minutes of data	C0

Detailed design docs in the repo (`docs/car-rental/`): architecture, architecture review, security, booking flow, service modes, channels, market reference, competitors, dev plan.

Sources (as of 10 Nov 2026; vendor pages and blogs are marketing, so values are treated as editable defaults)

- Competitors: HQ Rental Software, TSD, CarCloud AI agent, AiRentoSoft, Point of Rental agent, Hertz and UVeye, Smartcar
- Booking norms: "Or similar", ACRISS, grace periods, debit cards, overbooking
- UAE terms: RentDrive terms, PB Rent a Car, Dubai Small, deposits and the 30-day limit, km-limit court case, fines and Salik court case, Salik guide, UAE to Oman
- Qatar and Oman: Europcar Qatar terms, Qatar Living, Enterprise Muscat, Europcar Oman terms
- Chauffeur and insurance replacement: hourly chauffeur rates, Enterprise: who pays, UAE replacement car, replacement cover
- Channels: CarTrawler, OpenTravel endpoints, OneClickDrive listing, Turo/Getaround calendar sync, WhatsApp Flows, Google vehicle ads
- Compliance: UAE e-invoicing, Oman Fawtara, Stripe in the UAE, UAE data transfers
- Market signals: Sigvi, Latka